

一句话主线： 最终成功不是因为把文本更强行地注入分离器，而是把问题拆成先无条件分离，再用文本路由。端到端文本控制分离器推进 passthrough/对冲陷阱；PIT 分离器先把两个人拆开，文本只需要做二选，任务难度立刻降低。后续所有成功实验都围绕“更强的 separator、更稳的 router、更抗噪的训练分布”展开。

任务和评价指标

任务定义

本项目做的是两人混音场景下的 text-cued target speaker extraction：给定一个混音音频，以及目标说话人所说内容的文字线索，模型需要输出目标说话人的干净语音。

```
`mixture` + `cue text`
->
`target speech`
```

数据主要来自 LibriMix / Libri2Mix, 16 kHz, 2-speaker, min 模式。主干分离模型使用 BSRNN。集群路径长期使用：`/share/home/maiyou/tse1/wesep-real-tse`。

核心指标怎么读

p0.22p0.70

指标 | 含义

SI-SNRi | 相对原语音的 SI-SNR 提升，单位 dB，越高越好；接近 0 表示几乎没分离。本文最核心指标。
oracle ceiling | separator 已经给出的两个候选里，如果总能选对那一路，能达到的最高 SI-SNRi。它衡量 separator 上的 routing accuracy | router 是否选中了 oracle assignment 的那一路。它衡量文本路由能力。
accept | 输出 SI-SNRi 是否大于 dB 的比例，是一个粗略成功率。
wrong-text ablation | 把 cue 换成干扰说话人的文本后得到的 SI-SNRi，越低越说明模型真的受文本控制。

最容易误解的一点：路由器不直接产生 dB

当 separator 的两个候选已经固定时，router 只是在候选 0 和候选 1 之间选择。因此只要 routing accuracy 接近 100%，最终 SI-SNRi 就会接近 oracle ceiling。这也是为什么后期最大瓶颈逐渐变成 separator 本身，而不是 router。

总览：成功路线和最终结果

最终成功的几种方法

方法 | clean | WHAM 5 dB | WHAM 0 dB | 是否显式 ASR | 定位

SEP_LC + ASR 路由 | ~18.80 | 14.50 | 13.92 | 是 | 强基线，直观但依赖 ASR decoding
SEP_LC + MATCHER_WHAM | 18.80 | 14.53 | 14.14 | 否 | 当前最终最佳
SEP_LC + XATTN noisy router | 18.79 | 14.34 | 13.79 | 否 | 新方向，有潜力但未超过 matcher
SEP_LC + 蒸馏小 matcher | 18.40 | 13.66 | 12.59 | 否 | 小而快，但泛化不足

[表注] 目前真正成功的几条路线。单位均为 SI-SNRi dB。

一句话结论

- 路线 B (PIT separation + text routing) 是主线成功点。
- MATCHER_WHAM 是目前最好结果。它去掉了显式 ASR decoding，WHAM 0 dB 达到 14.14 dB，接近 oracle ceiling 14.26 dB。
- XATTN router 的价值在于证明 cross-attention 不是没用，而是必须训练在 separator-output domain 上。
- 蒸馏 student 的价值在于轻量化，缺点是对未见噪声泛化差。

阶段一：端到端文本直接控制分离器为什么失败

最初 idea

最自然的想法是：把文字 cue 编码成 embedding，再通过 cross-attention / FILM 注入 BSRNN，让 separator 直接输出目标说话人。这个方向看起来最“端到端”，也最像官方 speaker embedding TSE 的文本版本。

抽象流程是：

```
`mixture` + `text cue`
->
`text-conditioned separator`
->
`target speech`
```

为什么这一步值得做

如果成功，系统会非常优雅：不需要把两个人分开，也不需要后处理路由；文本直接驱动分离器。因此 R2--R8 反复尝试了从零训练、warm-start、冻结主干、解冻、dual-target、内容损失等变体。

实验结果

轮次 | 设置 | oracle | interferer | accept | 结论

R2 | 从零 + FiLM + swap | -0.029 | -0.029 | 0% | 忽略文本
R3 | 从零 + contrast + 强 FiLM | -0.043 | -0.043 | 0% | 忽略文本
R4 | warm + 冻结主干 | -0.463 | -0.480 | 17.7% | 会分离但乱猜
R5 | warm + 解冻 + dual-target | -0.0002 | -0.0002 | 0% | 搞成 passthrough
R6 | warm + 冻结 + dual-target | -0.047 | -0.051 | 0.75% | cue 盲
R7 | warm + 解冻 + 内容损失 w=1 | -0 | -0 | 0% | 内容损失太弱
R8 | warm + 解冻 + 内容损失 w=10 | -0 | -0 | 0% | 对冲线不过

[表注] 端到端文本条件化分离器 R2--R8 结果。

失败原因

这个阶段最重要的收获不是某个模型失败，而是识别出 passthrough / 对冲 陷阱。当模型还会根据文本判断目标是谁时，直接输出混音往往比瞎猜某个说话人更安全。对于 SI-SDR、内容 MSE、CTC-to-target 这类“输出 vs target”的损失，混音是一个很容易达到的局部最优。

诊断结果显示：

- 文本 embedding 本身可分，oracle 和 interferer 文本并不是混在一起。
- cross-attention / FILM 模块本身有响应，不同文本会导致中间调制量变化。
- 但最终音频输出变化很小，说明主干要么把调制冲掉，要么直接学 passthrough。

这一阶段的代码和文件

p0.30p0.62

文件/位置 | 作用

`examples/audio/librimix/cluster/train\_text\_xattn\_\*` | 各种文本注入 BSRNN 的 Slurm 训练脚本。
`examples/audio/librimix/cluster/eval\_swap\_xattn\_\*` | swap/interferer 文本评估脚本，用来判断 cue 是否真的生效。
`OPTIMIZATION\_JOURNEY.md` | 记录 R2--R8 的失败过程和诊断结论。

导师复述版

端到端文本控制分离器失败，不是因为文本 embedding 完全没信息，而是训练目标给不出 bootstrap 梯度。模型在还会听懂文本时，输出混音是一个低风险策略；一旦进入 passthrough，对调 loss 权重也很难救回来。因此下一步必须拆任务。

阶段二：路线 B，PIT 分离 + 文本路由

核心 idea

把原任务拆成两个简单任务：

- 无条件 PIT separation：不看文本，先把两个人分成两路。
- text routing：用文本 cue 在两路输出中选目标。

流程变成：

```
`mixture`
->
`PIT separator`
->
`candidate 0, candidate 1`
->
`text router`
->
`target`
```

为什么这一步能绕过 passthrough

PIT separation 要求输出两个 clean sources，而不是一个目标。它的训练目标天然不允许“输出混音就过关”，因此避开了端到端 TSE 的对冲陷阱。文本路由则被降维成二选一：只要知道哪一路说的内容更像 cue，就可以选对。

主要代码改动

p0.30p0.62

文件/位置 | 改动意义

`wesep/bin/train\_sep.py:1` | 新增 standalone 2-speaker PIT separator trainer。
`wesep/bin/train\_sep.py:104` | 实现 SI-SNR 和 PIT loss，两种 speaker permutation 取更好的一种。
`wesep/bin/train\_sep.py:133` | `warm\_sep`：从官方 speaker-embedding checkpoint 迁移 matching shape 的权重。
`wesep/bin/train\_sep.py:159` | 支持 `--resume` 和 `--warm\_sep`，方便 clean -> noisy -> bigger/longer 逐步训练。
`wesep/bin/eval\_compare.py:70` | 统一评估 ASR route、matcher route、oracle ceiling 和 noisy 条件。

ASR 路由结果

早期 route B 使用 ASR route：两路 separator 输出分别用 wav2vec2 ASR 转录，再和 cue text 比 CER，选择 CER 更低的一路。

在 clean test 上，ckpt150 达到：

- routed SI-SNRi: 17.54 dB
- accept: 99.2%
- routing accuracy: 99.5%
- wrong-text ablation: -37.9 dB

阶段结论

这一步是项目转折点：从 R2--R8 的 ≈0 dB 直接跳到 17 dB 以上。它证明了“文本线索是可用的”，问题在于端到端训练方式，而不是任务本身无解。

阶段三：去掉显式 ASR，训练 MATCHER

为什么要做 matcher

ASR route 很强，但它依赖显式转文字。下一步自然问题是：

能不能不先把语音转成文字，而是直接学习“语音内容”和“文本”的匹配？

这就是双塔 matcher 的动机。

重要澄清：这里真的去掉 ASR 了吗？

准确说：去掉的是 ASR decoding，不是完全去掉 ASR-pretrained representation。

在代码中，matcher 的音频塔仍然使用 torchaudio.pipelines.WAV2VEC2_ASR_BASE_960H 作为冻结特征提取器。但是推理时不再输出文字，也不计算 CER；只输出 audio embedding，与 text embedding 做相似度。

模型结构

- 音频塔：冻结 wav2vec2 -> mean pooling -> projection。
- 文本塔：字符级 embedding -> BiGRU -> projection。
- 损失：batch 内 InfoNCE，正确配对为正样本，其余为负样本。

代码位置

p0.30p0.62

文件/位置 | 作用

`wesep/bin/train\_matcher.py:46` | `ATData`：读取 source audio 和对应文本，可加噪训练。
`wesep/bin/train\_matcher.py:121` | `AudioEnc`：wav2vec2 音频特征 + projection。
`wesep/bin/train\_matcher.py:134` | `TextEnc`：字符级 BiGRU 文本塔。
`wesep/bin/train\_matcher.py:210` | 对称 InfoNCE / cross entropy 训练目标。
`wesep/bin/eval\_matcher.py:84` | matcher routing 与 wrong-text ablation。

matcher 训练结果

run | epochs | final loss / batch acc | 训练音频

MATCHER_V1 | 60 | 1.02 / 0.70 | clean source
MATCHER_NOISY | 60 | 1.60 / 0.54 | synthetic noise
MATCHER_WHAM | 80 | 1.29 / 0.61 | real WHAM noise

[表注] matcher 训练日志。batch acc 是 1-of-32 retrieval proxy，不等于最终 2-way routing accuracy。

优劣

优点：

- 不需要显式 ASR 转录，推理链路比 ASR route 简洁。
- 在 WHAM 噪声下与 ASR route 打平或更好。
- 双塔结构可复用：同一个 matcher 可以服务不同 separator。

缺点：

- 仍依赖 wav2vec2 ASR 预训练特征，不是完全摆脱 ASR 依赖。
- 双塔交互较浅，细粒度词级对齐能力可能不如 cross-attention。
- 对口音的真实鲁棒性还没有专门 benchmark，只能根据 wav2vec2 预训练和噪声实验做推断。

阶段四：抗噪训练，真正把 WHAM 0 dB 做起来

为什么要抗噪

clean-trained separator 和 matcher 在 clean 上很好，但一加噪声就崩。这说明噪声的主要伤害不只是 router 识别错，而是 separator 自身 oracle ceiling 大幅下降。

separator 优化阶梯

模型 | clean | WHAM 5 dB | WHAM 0 dB | 说明

OLD nr6 / 6s | 18.30 | 13.76 / 13.54 | 12.64 / 12.54 | 早期噪声模型
BiG nr8 / 6s | 18.60 | 14.34 / 14.51 | 13.63 / 13.29 | 加深 BSRNN
LC nr8 / 10s | 18.80 | 14.53 / 14.50 | 14.14 / 13.92 | 最终 separator

[表注] separator ladder，5 dB / 0 dB 中为 matcher route / ASR route；oracle ceiling 略高。

关键训练策略

- 从 clean separator 练到 real WHAM noise。
- chunk 从 6 s 加长到 10 s，让模型在低 SNR 下看到更长上下文。
- WHAM final 使用 SNR -5 到 20 dB 的更宽训练范围。

相关代码位置

p0.30p0.62

文件/位置 | 作用

`wesep/bin/train\_sep.py:39` | `SepData` 支持 `noise\_prob`/`snr\_min`/`snr\_max`/`noise\_dir`。
`wesep/bin/train\_sep.py:95` | 训练时按随机 SNR 混入噪声。
`wesep/bin/train\_sep.py:164` | `--num\_repeat` 控制 BSRNN 深度。
`wesep/bin/train\_sep.py:156` | `--chunk` 控制训练片段长度。
`wesep/bin/eval\_compare.py:74` | 评估时读取 held-out WHAM `tt` 噪声 split。

阶段结论

抗噪训练是第二个决定性进展。clean route 已经接近 ceiling 后，继续提升主要靠 separator。WHAM 0 dB 下，最终 MATCHER_WHAM 达到 14.14 dB，oracle ceiling 约 14.26 dB，说明路由由几乎已经不是主要瓶颈。

阶段五：轻量化蒸馏 student matcher

为什么要蒸馏

teacher matcher 仍然运行 wav2vec2，参数量大、推理成本高。蒸馏目标是让一个小 CNN 学到 teacher 的 audio embedding 空间，从而去掉 wav2vec2 依赖。

模型和损失

训练脚本 `wesep/bin/train\_matcher\_distill.py` 中写明：

```
= (1 - cos('student', 'teacher'))
+ lambda 'InfoNCE('student audio', 'text')
```

p0.30p0.62

文件/位置 | 作用

`wesep/bin/train\_matcher\_distill.py:30` | `StudentAudioEnc` 小 CNN audio encoder。
`wesep/bin/train\_matcher\_distill.py:39` | 使用 GroupNorm；避免 BatchNorm 在 batch=1 时崩。
`models/train\_matcher\_distill.py:138` | 加载 wav2vec2 teacher。
`wesep/bin/train\_matcher\_distill.py:213` | 保存 `kind=student\_cnn`，eval 自动识别。
`wesep/bin/diag\_student.py:2` | 诊断 student 在 `train/eval` 模式下 embedding 是否一致。

结果

router | params | clean | WHAM 5 dB | WHAM 0 dB | 结论

teacher wav2vec2 | 95M | 18.80 | 14.53 | 14.14 | 最强但重
student CNN + GroupNorm | 0.76M | 18.40 | 13.66 | 12.59 | 小、快，但掉点

[表注] 蒸馏 matcher 结果。

优劣

优点：

- 参数量约为 teacher 的 1/125。
- 不需要 wav2vec2，部署潜力最大。
- clean 下只落后 teacher 约 0.4 dB。

缺点：

- 在 WHAM 0 dB 落后 teacher 约 1.5 dB。
- 对未见噪声类型泛化差，Gaussian stress test 下明显崩。
- 说明小 CNN 学到了 WHAM-specific spectral shortcuts，还没有学到足够通用的语音内容表征。

阶段六：Cross-attention router

为什么重新尝试 cross-attention

双塔 matcher 很强，但它只在最终 embedding 层做相似度。cross-attention 的潜在优势是让 text tokens 直接 attend 到 audio frames，可能学到更细粒度的词级/帧级对齐。

第一版：clean-source XATTN router

第一版训练方式是让 router 在两个 clean source 中选目标。这个设置训练简单，但评估时 router 看到的是 separator 输出，而不是 clean source，因此存在明显 domain gap。

条件 | SI-SNRi | accept | routing acc | 结论

clean eval | 9.019 | 83.0% | 83.3% | 有信号但远不够
WHAM 0 dB | 5.237 | 80.3% | 80.0% | domain gap 很明显

[表注] clean-source XATTN router 结果。

第二版：训练在 SEP outputs 上

关键改进：训练时也先把 mixture 送进 frozen SEP_LC，得到两个 separated candidates。然后用 clean target source 计算两路候选的 SI-SNR，选择更好的一路作为 oracle label。

```
`mixture`
->
`frozen SEP_LC`
->
`candidate 0, candidate 1`
->
`oracle SI-SNR label`
->
`XATTN router`
```

这样训练分布和评估分布一致，解决第一版最大的 domain gap。

第三版：noisy SEP candidates

进一步把训练 mixture 加 WHAM train noise，SNR 在 0--5 dB 随机，训练 router 直接适应 noisy separator outputs。这一步不是盲目多训，而是改变训练分布，让它看见最难的 0 dB/5 dB 场景。

XATTN 关键代码

p0.32p0.60

文件/位置 | 作用

`wesep/bin/train\_xattn\_router.py:50` | clean-source router 数据集，后验证存在 domain gap。
`wesep/bin/train\_xattn\_router.py:156` | 第一版 `XAttNRouter`。
`wesep/bin/eval\_xattn\_router.py:156` | 输出 XATTN route SI-SNRi、routing accuracy、oracle ceiling。
`wesep/bin/train\_xattn\_router\_sep.py:64` | SEP-output router 数据集，可加入 WHAM 噪声。
`wesep/bin/train\_xattn\_router\_sep.py:164` | `SepXattNRouter`。
`wesep/bin/train\_xattn\_router\_sep.py:176` | `base\_scale` 初始化 matcher dot-product base。
`wesep/bin/train\_xattn\_router\_sep.py:180` | 从 `MATCHER\_WHAM` 初始化 audio/text encoders。
`wesep/bin/train\_xattn\_router\_sep.py:185` | `--router\_init`：从已有 SEP-router checkpoint 继续 noisy fine-tune。
`wesep/bin/train\_xattn\_router\_sep.py:209` | score = matcher base score + cross-attention residual。
`wesep/bin/eval\_xattn\_router\_sep.py:158` | 评估 routed SI-SNRi / accept / routing accuracy。

XATTN 结果

router checkpoint | clean | WHAM 5 dB | WHAM 0 dB | 说明

clean-source XATTN | 9.02 | -- | 5.24 | train clean source, eval separator output, domain gap 大
SEP-XATTN ep10 | 18.53 | 14.19 | 13.10 | 修正大部分 domain gap
SEP-XATTN noisy ep20 | 18.69 | 14.35 | 13.47 | later epoch 不是最佳
SEP-XATTN noisy ep11 | 18.79 | 14.34 | 13.79 | 最佳 XATTN checkpoint
MATCHER_WHAM | 18.80 | 14.53 | 14.14 | 仍是最终冠军

[表注] cross-attention router 与 matcher 比较。单位为 SI-SNRi dB。

XATTN 的优劣

优点：

- 证明 cross-attention 有价值，只要训练分布对齐。
- noisy SEP candidates 让 WHAM 0 dB 从 13.79 提升到 13.79。
- clean 上达到 oracle ceiling，wrong-text ablation 很低，说明文本控制真实存在。

缺点：

- 训练复杂，需要在线跑 frozen separator。
- 需要 early stopping：epoch 11 比 epoch 20 更好，说明盲目多训会迁移。
- 目前仍未超过简单的 MATCHER_WHAM。

关于口音/语音鲁棒性的判断

目前项目已有噪声鲁棒性实验，但没有专门的 accent benchmark。因此只能做基于结构的判断：

- ASR route 对口音风险最高，因为它依赖完整转录和 CER。口音导致 ASR 错训时，路由可能错。
- MATCHER_WHAM 不输出文字，可能比 ASR decoding 更稳；但它仍使用 wav2vec2 ASR 预训练特征，口音做差。
- XATTN router 同样使用 wav2vec2 特征，不是天然解决口音问题；不过它可以通过 noisy/accnt SEP candidates 继续训练。
- Student CNN 是否抗口音不确定，因为小模型容易学训练集捷径，需要专门数据验证。

建议后续做一个小型 accent stress test：

- 找 CommonVoice / L2-Arctic / accented LibriSpeech 子集。
- 构造两人 mixture，保留文本 cue。
- 固定 SEP_LC，比较 ASR route、MATCHER_WHAM、XATTN noisy router 的 routing accuracy。
- 分别报告 clean accent、WHAM 5 dB accent、WHAM 0 dB accent。

目前最有希望的优化空间

优先级 1：继续增强 separator

当前 MATCHER_WHAM 在 WHAM 0 dB 为 14.14 dB，而 oracle ceiling 约为 14.26 dB。也就是说 router 只剩约 0.12 dB 空间，主要瓶颈已经是 separator。如果想要显著超过 14 dB，优先考虑：

- 更强 separator backbone，例如 TF-GridNet、SepFormer、MossFormer 类结构。
- 更多真实噪声，更多 room/reverb augmentation。
- checkpoint averaging 或 EMA。
- train-360 扩大训练数据。

优先级 2：XATTN router 做 validation early stopping

noisy XATTN 的最佳 checkpoint 是 ep11，而 ep20 变差。这说明它不适合按 epoch 最后一个 checkpoint 取结果，应该固定一个 validation set：

- 指标用 WHAM 0 dB routing accuracy 或 routed SI-SNRi。
- 每个 epoch 后评估一个小 validation subset。
- 自动保存 best checkpoint。

优先级 3：matcher + XATTN score fusion

当前 matcher 稳，XATTN 有额外细粒度交互信号。可以不替换 matcher，而是融合：

```
`score`
= `matcher_score`
+ `xattn_residual`
```

这条路的好处是风险低：matcher 作为强 base，XATTN 只提供修正项。事实上 `train\_xattn\_router\_sep.py` 里已经有类似思想：
`base\_scale \* base + residual`。
后续可以把 base score 直接换成 frozen MATCHER_WHAM 的分数，再只训练 residual。

优先级 4：Student 用混合噪声蒸馏

Student 最大短板是泛化。下一版蒸馏应加入：

- WHAM + Gaussian + babble + DNS/noise bank。
- 更宽 SNR，例如 -5 到 20 dB。
- separated-output domain 蒸馏，而不仅是 clean/noisy source audio。

优先级 5：口音鲁棒性训练

如果导师关心 accent，下一步最好不是直接声称“抗口音”，而是做实验：

- 先评估：ASR route vs matcher vs XATTN。
- 再训练：把 accented speech 加入 matcher / XATTN router 的训练或 fine-tune。
- 报告 routing accuracy，而不仅是看 SI-SNRi，因为 separator 和 router 影响会混在一起。

按时间线复述给导师

可以这样讲：

- 最初尝试端到端文本条件化分离器，用 cross-attention 和 FiLM 把文本注入 BSRNN。但 R2--R8 都接近 0 dB。诊断发现模型容易输出混音，形成 passthrough/对冲最优解。
- 因此改成路线 B：先用 PIT 做无条件两人分离，再用文本路由。这个拆解绕过 passthrough，clean 上立刻达到约 17.5 dB，证明任务本身可行。
- ASR route 虽然强，但依赖显式转文字，于是训练双塔 matcher，用 wav2vec2 特征和字符级文本塔做 InfoNCE。这去掉了 ASR decoding，并在 WHAM 噪声下达到最终最好结果。
- 噪声实验发现 clean-trained 模型不抗噪，瓶颈主要是 separator ceiling。因此连续做 real WHAM noise training、加深 BSRNN、加长 chunk，最终得到 SEP_LC。

- 为了轻量化，做了 CNN student 蒸馏，证明小模型可行，但未见噪声泛化不足。
- 最后重新尝试 cross-attention router，第一版用 clean source 训练，测试在 separator output 上，有 domain gap。

改成训练时也过 SEP_LC，并用 oracle SI-SNR assignment 产生 label 后大幅提升；再加入 noisy SEP candidates，WHAM 0 dB 提到 13.79，但仍未超过 MATCHER_WHAM。

复现实验与 checkpoint 索引

p0.32p0.60

对象 | 路径/说明

最终 separator | `examples/audio/librimix/exp/SEP\_LC/models/ckpt\_100.pt`
最终 matcher | `examples/audio/librimix/exp/MATCHER\_WHAM/models/matcher\_80.pt`
最佳 XATTN noisy router | `examples/audio/librimix/exp/XATTN\_ROUTER\_SEP\_NOISY\_FT\_0\_5/models/xattn\_router\_sep\_11.pt`
separator 训练脚本 | `wesep/bin/train\_sep.py`
student 训练脚本 | `wesep/bin/train\_matcher.py`
student 蒸馏脚本 | `wesep/bin/train\_matcher\_distill.py`
XATTN SEP-router 训练脚本 | `wesep/bin/train\_xattn\_router\_sep.py`
统一比较评估 | `wesep/bin/eval\_compare.py`
XATTN SEP-router 评估 | `wesep/bin/eval\_xattn\_router\_sep.py`

关键集群任务

p0.18p0.30p0.42